SENTINEL

Análise Técnica de Projeto

Observability & FinOps Platform para Kubernetes

Versão analisada: v0.10.20 • Abril de 2026

1. Resumo Executivo

O Sentinel é uma plataforma open-source de observabilidade e FinOps para Kubernetes, desenvolvida como projeto pessoal por Marcel Boccato. A versão analisada é a v0.10.20, atualmente em fase de pré-lançamento público, previsto para a versão 1.0.

O projeto demonstra maturidade de engenharia consistente com profissionais sênior de plataforma: separação de responsabilidades bem definida, princípio de resiliência (deterministic-first), documentação técnica honesta sobre limitações e roadmap estruturado com critérios de conclusão claros.

Veredicto geral: projeto sólido, com arquitetura coerente e filosofia de produto bem definida. Tem brechas reais antes do lançamento público — principalmente em segurança e cobertura de testes — mas nada que comprometa a viabilidade como ferramenta comunitária.

Dimensão	Avaliação
Arquitetura & Design	✅ **Forte**
Qualidade de Código Go	✅ **Boa**
Cobertura de Testes	⚠️ **Parcial (25 testes)**
Segurança	⚠️ **Gaps críticos pré-v1.0**
Documentação	✅ **Acima da média**
Roadmap & Planejamento	✅ **Maduro**
Posicionamento de Mercado	✅ **Nicho bem definido**

Prontidão para v1.0 Público	⚠️ **~65% do caminho**

2. Arquitetura

2.1 Visão Geral

O Sentinel adota uma arquitetura em duas camadas, separadas por responsabilidade e acoplamento mínimo. Uma analogia útil: pense no Go Agent como uma UTI autônoma — coleta dados, monitora sinais vitais e toma decisões determinísticas independente de qualquer recurso externo. O LLM Agent é o médico especialista chamado opcionalmente: se ele não está disponível, a UTI continua funcionando.

Camada	Responsabilidade	Tecnologia
Go Agent (porta 8080)	Coleta, persistência, dashboard, API REST	Go 1.23, client-go, net/http, slog, embed
LLM Agent (opcional)	Análise narrativa, geração de runbooks	Python + harness + qualquer LLM (Claude, Gemini…)
PostgreSQL	Persistência 3 camadas: raw / hourly / daily	Pod no namespace sentinel (via Helm)
Dashboard	UI interativa embutida no binário	HTML + CSS + Chart.js (//go:embed)
Harness	Gatekeeper de segurança para outputs do LLM	Python validador_saida.py (16 testes)

2.2 Pontos Fortes da Arquitetura

- Deterministic-first: o sistema funciona completamente sem LLM. A IA é camada opcional, não dependência central.
- Embed do dashboard no binário: elimina problemas de path de assets em deploy, simplifica o Dockerfile multi-stage.
- Separação collection / calculation / presentation: a UI não faz cálculos; o backend não toma decisões de renderização.
- Helm chart completo: PostgreSQL sobe automaticamente como pod no namespace sentinel, sem pré-requisitos locais para deploy em cluster.
- Config externa via ConfigMap: thresholds.yaml montado via Helm — operadores podem ajustar limites sem recompilar.
- Harness de segurança: validador_saida.py bloqueia comandos destrutivos (rm -rf, kubectl delete, DROP TABLE) antes que o LLM execute qualquer ação.

2.3 Gaps Arquiteturais

- Single-cluster apenas: não há suporte a múltiplos clusters. A própria roadmap posiciona isso como post-1.0, o que é honesto, mas é uma limitação relevante para times maiores.
- Metrics API como única fonte: sem suporte a Prometheus como backend alternativo. Reduz a profundidade de métricas disponíveis (sem histogramas, sem P99 de latência).

- Sem autenticação no Go Agent: qualquer pessoa com acesso à rede lê /api/waste, /api/incidents e dados de custo. O M7 prevê BasicAuth/token, mas é um gap crítico pré-release público.
- PostgreSQL sem redundância: o banco roda como pod single-instance. Restart do pod implica perda dos últimos dados não-agregados. Sem PVC na configuração padrão.

3. Análise de Código

3.1 Go Agent

O agente Go é o núcleo do projeto. A refatoração feita na v0.10.17 reduziu o main.go de ~2.282 para ~220 linhas — sinal concreto de maturidade: o desenvolvedor reconheceu o acúmulo de responsabilidades e agiu. É o equivalente a um encanador que para a obra, reorganiza as tubulações antes de continuar, em vez de só empurrar mais canos.

Pacote	Responsabilidade
pkg/api	HTTP handlers, tipos (PodStats, WasteEntry, Incident), middleware, Swagger
pkg/k8s	Client Kubernetes + wrappers da Metrics API
pkg/store	PostgreSQL: schema, agregação, retenção
pkg/incidents	Carregamento de thresholds + detecção determinística
main.go	Bootstrap, goroutine do collector, servidor HTTP (~220 linhas)

Pontos positivos

- Uso de slog (structured logging): saída de logs com campos tipados (component, level, msg, err). Padrão moderno do ecossistema Go 1.21+.
- OpenAPI embutido via //go:embed: o spec openapi.yaml é compilado dentro do binário. O Swagger UI é servido pelo próprio agente sem dependências de build externas.
- Regressão linear OLS para forecast de custo: implementação própria com banda de confiança ±1.5σ. Simples e suficiente para o caso de uso.
- Thresholds como única fonte de verdade: config/thresholds.yaml lido tanto pelo Go agent quanto montado via ConfigMap. Sem valores hardcoded dispersos.
- AuthMiddleware implementado: a v0.10.18 adicionou AuthEnabled/AuthToken para sincronização multi-instância (entre instâncias opencode e gemini). O mecanismo existe, só precisa ser exposto como configuração principal no M7.

Pontos de atenção

- Collector goroutine sem circuit breaker: se a Metrics API do cluster ficar intermitentemente indisponível, o collector falha silenciosamente (ou em loop). Não há backoff exponencial documentado.
- Retenção de dados sem PVC padrão: o PostgreSQL sobe como pod mas o Helm chart não configura PersistentVolumeClaim por padrão. Restart do pod = perda dos dados raw (24h de coleta).
- Estimativa de custo sem preço real configurável: o forecast usa unidades de 'custo' sem mapear para moeda real. Para FinOps sério, isso precisa de um campo de preço por mCPU/MiB configurável.

3.2 Harness & Segurança do LLM

O harness/validador_saida.py com 16 testes unitários é o componente mais defensivo do projeto. Ele atua como um porteiro: nenhum output do LLM chega ao sistema sem passar pela validação. As regras são claras: bloqueia comandos destrutivos, exige ## Resumo Executivo no relatório e rejeita conteúdo abaixo de 100 caracteres.

> Isso é uma decisão de design madura: não confiar no LLM para tomar decisões seguras, e colocar um validador determinístico na frente. O conceito é correto e bem executado para a fase atual.

3.3 Cobertura de Testes

Componente	Testes	Cobertura
Go Agent (5 pacotes)	25 testes	Parcial — collection + waste
Harness Python	16 testes	Boa — regras do validador
Dashboard JS	0 testes	Ausente
Helm chart	0 testes	Ausente (sem helm test)
Integração / E2E	0 testes	Ausente

41 testes no total é razoável para um projeto solo em fase 0.x. O gap principal é a ausência de qualquer teste no dashboard JS — que tem ~41% do código — e de testes de integração básicos verificando que /api/incidents retorna o formato esperado após uma coleta real.

4. Documentação

4.1 README

O README está bem estruturado e honesto sobre o estado do projeto. Cobre: proposta de valor, arquitetura ASCII, stack, pré-requisitos, setup (Helm + local), endpoints, features do dashboard e changelog detalhado por versão. A progressão do changelog é particularmente útil — mostra a evolução do pensamento de design da v0.1 (multi-agentes) até v0.10.20 (dashboard v2 no-scroll).

Destaque: o README não promete o que o projeto não entrega. A seção 'Why Sentinel?' é honesta sobre o target (times sem budget para Kubecost/Datadog) e não faz comparações infladas.

4.2 ROADMAP

O ROADMAP.md é um dos ativos mais maduros do projeto. Cada milestone tem: objetivo claro, tabela de deliverables com status individual, critério de conclusão explícito e dependências mapeadas. Isso é raro em projetos solo — a maioria tem um TODO.md informal ou nada.

- Princípios de produto explícitos: 'Se o LLM cair, Sentinel continua útil', 'Se o dashboard falhar, a API deve ser usável'. Isso guia decisões de design de forma consistente.
- Categorização Core / Support / Luxury: deixa claro o que é essencial vs. o que é desejável. Evita scope creep.
- Gap: o backlog do M4 (Online Boutique lab) aparece repetidamente no ROADMAP mas ainda não iniciado. Para v1.0, um demo real com cluster degradado faria diferença significativa na adoção.

4.3 OpenAPI / Swagger

A presença do openapi.yaml embutido no binário com Swagger UI acessível em /docs é um diferencial real. Qualquer desenvolvedor que clonar o projeto pode explorar a API sem ler documentação externa. Todos os endpoints estão cobertos segundo o ROADMAP M3.

5. Posicionamento de Mercado

5.1 Nicho

O Sentinel ocupa um espaço genuinamente sub-servido: times pequenos que rodam Kubernetes mas não têm budget nem equipe para operar Kubecost, Datadog ou Harness. A proposta 'zero Prometheus required' é o diferencial mais concreto — elimina a maior barreira de entrada de observabilidade K8s.

Ferramenta	Requer Prometheus?	FinOps nativo?	Standalone?
Sentinel	Não	Sim	Sim
Kubecost	Sim	Sim	Não
Grafana + kube-state	Sim	Parcial	Não
Lens	Não	Não	Sim
k9s	Não	Não	Sim

5.2 Diferencial Competitivo

- Único com grau de eficiência por namespace (A→F + UNMANAGED) sem dependência de stack externo.
- Forecast de custo com OLS + banda de confiança acessível sem configuração adicional.
- Binário único: embed de dashboard + OpenAPI + assets. Deploy simples, auditável.
- LLM como camada opcional com harness de segurança — posicionamento honesto da IA, sem hype.

5.3 Riscos de Mercado

- Kubecost tem tier gratuito que pode cobrir parte do nicho com mais depth de métricas.
- OpenCost (CNCF) está ganhando tração como padrão de FinOps K8s open-source. Sentinel precisará de diferencial claro quando o OpenCost amadurecer.
- Dependência de Metrics API limita a granularidade: sem histogramas, sem traces, sem RED metrics por serviço. Para SRE sério, isso é uma lacuna visível.

6. Segurança

⚠️ Esta seção contém os gaps mais críticos antes do lançamento público como v1.0.

6.1 Go Agent — Exposição de API

- Sem autenticação padrão: /api/waste, /api/incidents, /api/history e /api/efficiency expõem dados potencialmente sensíveis (custo por pod, nomes de deployments, padrões de uso) sem qualquer controle de acesso por padrão.
- AuthMiddleware existe mas não está ativo por padrão: o mecanismo AuthEnabled/AuthToken foi implementado na v0.10.18 para sincronização interna. Precisa ser exposto como configuração obrigatória no Helm chart antes do release público.
- Sem TLS: o agente serve em HTTP puro. Para qualquer ambiente que não seja localhost/Minikube, isso é um risco real.
- CORS não documentado: o comportamento de CORS do net/http padrão precisa ser explicitado — especialmente se o dashboard for acessado de origens externas.

6.2 Harness — Pontos Positivos

- Bloqueio de comandos destrutivos: rm -rf, kubectl delete, DROP TABLE, fork bombs. Implementação correta e testada.
- Validação de estrutura mínima do relatório: rejeita outputs sem ## Resumo Executivo e abaixo de 100 chars.
- 16 testes unitários cobrem os casos principais de bloqueio.

6.3 Recomendações Críticas Pré-v1.0

- Ativar autenticação por padrão no Helm chart (AuthEnabled=true com token gerado no install).
- Adicionar opção de TLS ou documentar explicitamente que o agente deve ficar atrás de ingress com TLS.
- Adicionar network policy no Helm chart restringindo acesso ao agente ao namespace sentinel.
- Definir Security Policy (SECURITY.md) com processo de reporte de vulnerabilidades — o arquivo existe no repo mas precisa de conteúdo real.

7. Análise do Roadmap

7.1 Estado Atual vs. v1.0

Milestone	Status	Gap para v1.0
M1 — Core	✅ Done (v0.10.1)	Nenhum
M2 — FinOps	✅ Done (v0.10.15)	Preço por unidade configurável
M3 — Incidents	✅ Done (v0.10.20)	Correlação entre eventos
M4 — Lab	Não iniciado (v0.11)	Demo real é essencial para adoção
M5 — Self-obs	✅ Done (integrado M1)	Nenhum
M6 — LLM layer	~20% (v0.12)	Modo degradado sem LLM
M7 — v1.0 prep	Não iniciado (v1.0)	Auth, docs, contratos estáveis

7.2 Estimativa Realista para v1.0

Com dedicação de hobby (não tempo integral), a estimativa conservadora para um v1.0 publicável é de 4 a 6 meses a partir de agora, assumindo:

- M4 (Online Boutique lab): 3-4 semanas de setup + documentação de cenários
- M6 (LLM layer degradado): 2-3 semanas para modo fallback funcional
- M7 (polish, auth, docs): 4-6 semanas — a parte que mais demora em projetos solo
- CI/CD básico (GitHub Actions: go test + helm lint): 1 semana

> O risco real do M7 não é o código — é a disciplina de parar de adicionar features e polir o que existe. 'Done criterion: another developer can clone, configure and run Sentinel without help' é o critério correto. Optimize para ele.

8. Recomendações Prioritárias

8.1 Críticas (pré-lançamento público)

#	Recomendação	Impacto
1	Ativar AuthEnabled=true por padrão no Helm chart com token auto-gerado	Segurança — crítico
2	Configurar PersistentVolumeClaim padrão para o PostgreSQL no Helm chart	Resiliência de dados
3	Adicionar GitHub Actions: go test ./... + helm lint em PRs	Qualidade — impede regressões
4	Documentar variáveis de ambiente no README (todas, com defaults)	Usabilidade para novos usuários
5	Adicionar preço por mCPU/MiB como variável configurável no forecast	Precisão do FinOps

8.2 Médio Prazo (v0.11 - v0.12)

- Implementar M4 (Online Boutique lab) com cenários documentados: normal, CrashLoop, OOMKill, resource waste. Isso é o 'proof of concept' que a comunidade vai querer ver antes de adotar.
- Adicionar backoff exponencial no collector goroutine para falhas da Metrics API.
- Circuit breaker simples para o banco: se o PostgreSQL ficar indisponível, o agente serve dados em cache da última coleta com flag de staleness no /health.
- Testes de integração básicos: um teste que sobe o agente em modo mock e valida os contratos de /api/incidents e /api/waste.
- Suporte a múltiplos namespaces no /api/efficiency com filtro por label (não só por nome).

8.3 Longo Prazo (v1.0+)

- Suporte a Prometheus como backend opcional: mantém a proposta standalone mas abre para clusters que já têm Prometheus, aumentando a profundidade de métricas.
- Plugin system para LLMs: ao invés de hardcodar endpoints de LLM, um registro de providers (Claude, Gemini, Ollama) com interface comum.
- Multi-cluster via configuração: context switching de kubeconfig para comparação entre clusters.
- Alertas push: webhook configurável para enviar incidentes críticos para Slack/Teams sem depender do dashboard estar aberto.

9. Valor como Projeto de Portfolio

O Sentinel demonstra um conjunto de competências que é exatamente o que recruiters e hiring managers de SRE/Platform Engineering procuram — e raramente veem juntas em um projeto solo:

Competência	Evidência no Projeto
Go + client-go	Agente standalone com Kubernetes Metrics API, 4 pacotes bem estruturados
Kubernetes profundo	InClusterConfig, Metrics API, Helm chart, ConfigMap, NodePort, RBAC implícito
FinOps engineering	Waste por pod/deployment, forecast OLS, grades A→F por namespace
SRE thinking	Deterministic-first, harness de segurança, /health endpoint, self-observability
Design de produto	ROADMAP com critérios de conclusão, princípios explícitos, changelog honesto
LLM/AI pragmático	LLM como camada opcional com harness de validação — não hype, mas engenharia

Como apresentar em entrevistas: 'Construí um observability platform standalone para Kubernetes que detecta waste e incidentes via regras determinísticas, sem dependência de Prometheus. Adicionei um LLM como camada opcional com um harness de segurança que valida e bloqueia outputs destrutivos antes de qualquer execução.' — isso abre 5 perguntas técnicas boas em qualquer painel de SRE.

9.1 O que Adicionar ao Portfolio

- Adicionar um GIF animado do dashboard no README: visualização imediata do produto aumenta significativamente o engajamento no GitHub.
- Publicar um blog post técnico (dev.to ou Medium) explicando a decisão de não usar Prometheus e como a regressão linear do forecast foi implementada.
- Submeter à CNCF Landscape como ferramenta de observabilidade — o projeto já tem a maturidade mínima necessária para listagem.
- Adicionar badge de CI/CD no README após implementar GitHub Actions — sinaliza projeto ativo e testado.

10. Conclusão

O Sentinel é um projeto de hobby com qualidade de projeto profissional. A filosofia de design é consistente e defensável, a arquitetura tem separação de responsabilidades real (não apenas nominal), e o ROADMAP demonstra que o autor pensa em produto, não só em código.

Os gaps são reais mas conhecidos: segurança pré-release, cobertura de testes do frontend, demo em ambiente real. Nenhum deles é surpresa — todos estão mapeados no próprio ROADMAP. Isso é um sinal de auto-consciência técnica que importa.

Para o lançamento v1.0 na comunidade, as prioridades são claras: ativar autenticação por padrão, implementar o M4 com Online Boutique para provar valor em cenário real, e polish de usabilidade para que outro desenvolvedor consiga rodar o projeto sem ajuda. Se esses três itens estiverem completos, o Sentinel tem condições reais de encontrar sua audiência na comunidade CNCF/SRE.

> ***"If the LLM goes down, Sentinel stays useful."***
> — Princípio de design do Sentinel

Análise gerada por Claude (Anthropic) • Abril de 2026